Practice DL: how my code works

Oct 24, 2026 · @Sanket

My second practice project. I went through 4 deep learning topics one step at a time: images (A), text (B), deploying a model as an app (C) and reinforcement learning (D). These are my notes on how each script works, what it got, and what broke along the way.

My rule for every step: run a simple baseline next to the model, so I know whether it actually learned something.

Setup and how to run

Everything ran on my MacBook (M-series), with PyTorch using the Mac GPU (`mps`). The scripts are in my DeepLearning folder with a `requirements.txt`.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Most scripts have a `--quick` flag. I always ran that first, because it uses a small part of the data and finishes in about a minute, so I know the code works before the long run.

step	file	data	time on my mac
A1	a1_cnn_cifar.py	CIFAR-10, 60k photos	~3 min
A2	a2_transfer.py	CIFAR-10 + ResNet18 weights	~10 min
B1	b1_bow_imdb.py	IMDB, 50k reviews	~30 s
B2	b2_embeddings.py	IMDB	~2 min
B3	b3_transformer.py	IMDB + DistilBERT	~12 min
C	c_btc_app.py	live Coinbase BTC price	runs until I stop it
D1	d1_qlearning.py	Gymnasium FrozenLake, Taxi	~5 s
D2	d2_dqn_cartpole.py	Gymnasium CartPole	~30 s

A1 - CNN from scratch (a1_cnn_cifar.py)

My CNN got **88.0%** on the 10k CIFAR-10 test photos, against 51.7% for an MLP and 40.1% for a linear model.

What the script does

1. `load_cifar()` downloads CIFAR-10 through torchvision, turns the whole dataset into tensors, normalises each colour channel, and keeps everything on the GPU. 45k photos go to training, 5k to validation, and the 10k test set is only used at the end.
2. `augment()` flips half the images and shifts each one by up to 4 pixels, so the CNN sees slightly different pictures every epoch.
3. Three models go through the same `train()` loop: `linear_model()` (one weight per pixel per class), `mlp_model()` (two hidden layers, but the image is still flattened) and `cnn_model()`.
4. The CNN is 3 `conv_block`s. Each block is two 3x3 convolutions with batch norm and ReLU, then max pooling halves the image (32 → 16 → 8 → 4). Then I average each channel and use one linear layer for the 10 classes.
5. `train()` uses AdamW with a OneCycle learning rate, and keeps the weights from the epoch with the best validation accuracy.
6. At the end it prints per-class accuracy, saves a confusion matrix plot, and saves `a1_cnn.pt` so A2 can reuse it.

Results

model	test acc	weights
just guessing	10.0%	-
linear	40.1%	30,730
MLP	51.7%	1,707,274
CNN	88.0%	289,194

The CNN has 6x fewer weights than the MLP but beats it by 36 points, because the same small filters slide over the whole image instead of every pixel getting its own weight. Cat was the hardest class (74%), mostly confused with dog.

A2 - Transfer learning with ResNet18 (a2_transfer.py)

Starting from a ResNet18 pretrained on ImageNet got **95.2%**, and with only 2,000 photos it got 87.8%, about the same as my CNN trained on all 45,000.

What the script does

1. It reuses `load_cifar`, `augment` and `cnn_model` from A1, and loads `a1_cnn.pt` to test my A1 CNN again as the baseline.
2. `make_prep()` undoes A1's normalisation, redoes it with ImageNet's mean/std, and resizes the images from 32x32 to 128x128, because ResNet was trained on bigger pictures.
3. **Frozen:** I remove ResNet's last layer, run every image through it once with `get_features()` (512 numbers per image), and train only a new linear layer on those features.
4. **Finetune:** I swap the last layer for a 10-class one and train the whole network for 3 epochs with a lower learning rate (5e-4), so I don't wreck the pretrained weights.
5. **Small data:** the same comparison with only 2,000 training photos, CNN from scratch vs finetuned ResNet.

Results

model	trained on	test acc
A1 CNN from scratch	45k	88.0%
frozen ResNet + new last layer	45k	85.9%
finetuned ResNet	45k	95.2%
CNN from scratch	2k	67.0%
finetuned ResNet	2k	87.8%

The frozen version trains only 5k weights and still almost matches my whole CNN, so ImageNet features already work for CIFAR. With 2k photos my CNN overfit badly (train 88%, val 68%).

B1 - Bag of words on IMDB (b1_bow_imdb.py)

Logistic regression on word counts plus word pairs got **90.6%** on 25k test reviews, but it still thought “this movie was not bad at all” was negative.

What the script does

1. `load_imdb()` downloads the 50k IMDB reviews once and saves them as `imdb.csv`. It tries the Hugging Face copy first and Stanford second (see problems). `clean()` removes the html `<br />` tags and lowercases the text.
2. `word_list_rule()` is my own baseline: count words from my GOOD list vs my BAD list and say positive if good ≥ bad.
3. Naive Bayes uses raw word counts from `CountVectorizer`.
4. The two logistic regressions use `TfidfVectorizer`. TF-IDF counts words, but words that show up in almost every review (“the”, “movie”) count less. The second one also uses word pairs (`ngram_range=(1, 2)`), so “not good” becomes its own feature.
5. It prints the biggest positive and negative weights, the weights of some “not” pairs, a few reviews it got wrong, and its guesses on my test sentences. You can pass your own with `--try`.

Results

model	test acc
always say positive	50.0%
my word list rule	70.5%
naive bayes	81.5%
logreg, single words	88.7%
logreg, words + pairs	90.6%

“not funny” got -3.15 even though “funny” is +0.86, so pairs help. But “bad” is -13.03 and “not bad” only +1.48, and since word order doesn’t matter, “not bad at all” still came out at 0.01.

B2 - Word embeddings + BiLSTM (b2_embeddings.py)

Neither neural model beat my B1 logreg: average embeddings got **87.4%** and the BiLSTM **87.1%**, against 90.6% for logreg.

What the script does

1. It reuses `clean` and `load_imdb` from B1. `build_vocab()` keeps the 30k most common words (seen at least 3 times). Index 0 is padding and 1 is “unknown word”.
2. `encode()` turns each review into 300 word ids, cutting longer ones and padding shorter ones.
3. `AvgEmbeddings`: every word gets a vector of 100 numbers (`nn.Embedding`). I average the vectors of a review, skipping padding, and one linear layer says positive or negative. Word order still doesn’t matter.
4. `BiLSTM`: the same embeddings, but an LSTM reads the review word by word, forwards and backwards. Then I take the max of each feature over all words.
5. `train()` uses Adam, clips gradients (LSTMs can blow up without it), and keeps the best epoch on validation (2.5k reviews held out).
6. `nearest_words()` finds the words whose embeddings are closest by cosine similarity.

Results

model	test acc
B1 logreg (baseline)	90.6%
avg embeddings	87.4%
BiLSTM	87.1%

With 22.5k reviews there isn’t enough data to learn 3M embedding numbers from scratch. The embeddings still learned sentiment: closest to “awful” were waste, worst, disappointment, boring, horrible. “actor” got random neighbours, because sentiment is the only thing the model was trained on.

B3 - DistilBERT (b3_transformer.py)

My finetuned DistilBERT got **91.8%** and beat logreg (90.6%) using only 8k reviews, but only after I stopped cutting off the end of the reviews. My first try got 88.2%.

What the script does

1. It retrains the B1 logreg on all 25k reviews, so every model is tested on the same 5,000 random test reviews.
2. It loads `distilbert-base-uncased-finetuned-sst-2-english`, which someone else already trained for sentiment on short movie sentences, and tests it without any training.
3. It loads plain `distilbert-base-uncased` with a new 2-class layer and finetunes it in `finetune()`: AdamW with lr 2e-5 (small, because the weights are already good), linear warmup, gradient clipping, and keeping the best epoch on 500 validation reviews. The best weights are kept on the CPU so they don't take GPU memory.
4. `encode()` turns text into tokens. Normally it keeps the first `--max-len` tokens. With `--head-tail` it keeps the first 128 tokens plus the end of long reviews.

Results (same 5,000 test reviews)

model	reviews trained on	try 1: 256 tokens	try 2: 384 head+tail
B1 logreg	25,000	90.6%	90.6%
SST-2 DistilBERT, no training	0	86.8%	88.5%
my DistilBERT finetune	4k / 8k	88.2%	91.8%

Try 2 command: `python b3_transformer.py --train-size 8000 --max-len 384 --head-tail`. Even the untrained SST-2 model went up 1.7 points just by reading the ending, so the verdict often sits at the end of a review. The "MISSING / UNEXPECTED" load report when it starts is normal: the new classifier layer starts random.

C - Live BTC app in Streamlit (c_btc_app.py)

The app works live, and it showed that my old 66% bounce rule only works if I sample the price fast: **61% at 3 s** but **42% at 8 s**.

What the script does

Run it with `streamlit run c_btc_app.py`. It opens in the browser at localhost:8501.

1. `init_state()` keeps everything in `st.session_state`: prices, the last 3 moves, the pending predictions, hits per model and the log table. If `prices.csv` from my realtime project is in the folder, `warm_start()` lets the logreg learn from it first.

2. `live()` is a `@st.fragment(run_every=interval)`, so only that part reruns every few seconds. Each time it calls `fetch_price()` (the Coinbase spot price, no key), or `fake_price()` when the offline checkbox is on.
3. `update()` is the same logic as step5 in my realtime project. Ticks where the price didn't move are skipped. When it moves, the last predictions get scored, the logreg learns from that move, and every model predicts again.
4. Models: majority (baseline), bounce (opposite of the last move), momentum (same as the last move) and `OnlineLogReg`, an `SGDClassifier` with `partial_fit` on the signs of the last 3 moves.
5. The page shows the price, each model's next prediction, a price chart (Altair, so the y axis doesn't start at 0), rolling accuracy over the last 100 moves vs a coin flip, and a table of recent predictions with ✓ and ✗.

Results

model	run 1: 8 s, 119 moves	run 2: 3 s, 119 moves
coin flip	50%	50%
majority	49%	52%
bounce	42%	61%
momentum	-	39%
online logreg	42%	52%

Sampled every 2-3 s, the Coinbase price flips between bid and ask, so "opposite of the last move" works. At 8 s that bounce is mostly gone. So my "model" was really a model of how Coinbase quotes the price. In run 1 the logreg just copied bounce from the old data. With a higher learning rate in run 2 it kept changing its mind and got 52%.

D1 - Q-learning (d1_qlearning.py)

Q-learning reached the FrozenLake goal **74.2%** of the time (random agent 0.9%), and on Taxi it scored **+7.9** per ride (random -767.7).

What the script does

There's no dataset in RL. The agent plays the game, gets rewards, and fills a table `Q[state, action]` with how much total reward it expects.

1. `q_learning()` plays many episodes. With probability epsilon it does a random action (exploring), otherwise the best action in its table. Epsilon starts at 1 and goes down to 0.05 over the first 80% of episodes.
2. After every step it updates the table:

```
Q(s,a) \leftarrow Q(s,a) + \alpha \left( r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right)
```

3. If the episode really ended there is no future reward, so the target is just the reward.
4. `evaluate()` plays 1000 episodes with the learned table (no exploring) and with random actions, for comparison.
5. `show_frozenlake_policy()` prints an arrow for the best action on every square, and at the end it shows one taxi ride step by step.

What it learned on FrozenLake

```
← ↑ ↑ ↑
← H → H
↑ ↓ ← H
H → ↓ G
```

On the start square it pushes into the wall. The ice is slippery, so you only go where you want 1/3 of the time. Pushing into the wall means a slip can only send you down or keep you in place, never into a hole. The training curve stays around 55% because it still takes random actions 5% of the time while learning. Newer gymnasium renamed Taxi-v3 to Taxi-v4, so `make_taxi()` tries v4 first.

D2 - DQN on CartPole (d2_dqn_cartpole.py)

DQN balanced the pole for the full **500 steps** in every test episode, but the training was really unstable on the way there.

What the script does

CartPole's state is 4 continuous numbers (cart position, cart speed, pole angle, pole angular speed), so a plain table doesn't work.

1. `my_rule()` is my hand-written baseline: push towards the side the pole is leaning or falling to.
2. `q_table()` is D1's Q-learning after `to_bins()` chops the 4 numbers into bins (3,600 states).

3. `dqn()` replaces the table with a small network, 4 → 128 → 128 → 2, that outputs a Q value for "push left" and "push right". Two tricks make it stable:
   - **Replay buffer:** every step goes into a memory of 50k steps, and the network learns from random batches of 64 old steps, not just the last one.
   - **Target network:** a copy of the network computes the targets and moves only 1% towards the real one after each step (`tau=0.01`).
4. Every 20 episodes it tests the greedy policy on 10 episodes and keeps the best version. It runs on the CPU, which is faster than the GPU for a network this small.

Results (avg steps over 100 test episodes, max 500)

agent	avg steps
random pushes	21.9
my 1-line rule	500.0
Q-table with bins	487.8
DQN (400 episodes, 20 s)	500.0

The greedy test score went 370 at episode 80, down to 43 at episode 200, then 500 at episode 340. The network can forget what it learned while it trains on new experience, which is why keeping the best version matters. My first settings (exploring for 10k steps, `tau=0.005`) got stuck around 300. Exploring for 5k steps with `tau=0.01` got to 500. Honestly, CartPole is easy if you know the physics, since my 1-line rule gets 500 too. The point is that DQN learned it from rewards alone.

Problems I ran into

Eight things broke or didn't work at first. Here is each one with the fix, in the order they happened.

step	what happened	how I fixed it
setup	`pip install` gave `zsh: command not found: pip` , then `python -m pip` said `No module named pip` . My new DeepLearning .venv was made without pip.	`python -m ensurepip --upgrade` . Since then I always use `python -m pip install ...` so it goes into the active venv.

step	what happened	how I fixed it
setup	VS Code popup: “Extension activation failed”.	Nothing. It didn’t affect running scripts in the terminal.
B1	The IMDB download from Stanford’s server failed with `URLError: [Errno 61] Connection refused`.	Download the same dataset from Hugging Face (needs `pandas` + `pyarrow`), with Stanford kept as a backup. Lesson: don’t depend on one link.
B2	Testing the BiLSTM on 25k reviews crashed with `RuntimeError: MPS backend out of memory`.	Predict in batches of 100 instead of 500 and call `torch.mps.empty_cache()`. When that still wasn’t enough, predictions fall back to the CPU.
B2	The same crash happened again, because I hadn’t saved the new file. The traceback still showed the old line number.	Save first (Cmd+S), and check with `wc -l` that the file has the right length.
B3	My first DistilBERT finetune (88.2%) was worse than logreg (90.6%).	Read 384 tokens and keep the start + end of long reviews, with 8k reviews: 91.8%.
C	The price chart started at 0, so BTC looked like a flat line. The “no move” counter also counted the old prices.csv (3,695). The new app opened on port 8502 because the old one was still running on 8501.	Altair chart with `zero=False`, reset the counter after the warm start, and stop the old app with Ctrl+C.
D2	My first DQN settings got stuck around 100-300 steps.	Explore for 5k steps instead of 10k and update the target network faster (`tau` 0.005 → 0.01).

What I learned

1. **Pretrained models are worth a ton of data.** ResNet on 2k photos matched my CNN on 45k. DistilBERT on 8k reviews beat logreg on 25k.
2. **Simple baselines are hard to beat.** Bag of words logreg beat both neural nets in B2, and a 1-line physics rule balanced CartPole perfectly.

3. **The data setup matters as much as the model.** Reading the end of reviews took DistilBERT from 88.2% to 91.8%. Changing the price interval from 3 s to 8 s turned bounce from 61% into 42%.
4. **Patterns from old data can stop working live**, so a deployed model has to keep checking its own accuracy.
5. **Deep learning training is noisy.** DQN went 370 → 43 → 500, and validation accuracy jumped around in every track. Keeping the best checkpoint on validation saved me every time.